A3 • TEXTE STRUCTURÉ & FIABILITÉ

Contrats JSON sous stress

Mesurer la fiabilité des sorties JSON d’un LLM et l’efficacité d’une stratégie de validation + retry.

Ruofan ZHAO, Zihao CHENG, Yao LIU

Pourquoi tester les “contrats JSON” ?

Risques observés

- Syntaxe invalide
- Champ obligatoire absent
- Mauvais type / mauvaise valeur
- Règle métier non respectée

Question de recherche

Comment améliorer la conformité des sorties JSON générées par un LLM ?

- Mesurer la conformité au premier essai
- Comparer 5 niveaux de complexité
- Comparer une stratégie de référence avec retry
- Quantifier coût, tentatives et latence

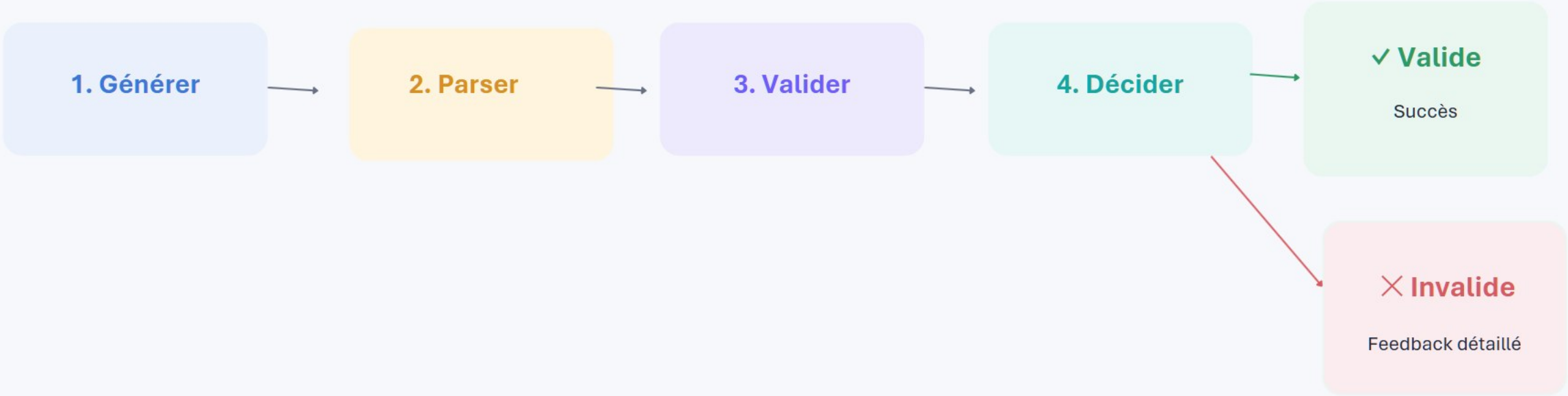
Architecture du projet



Conception des JSON Schemas



Validation des données



Protocole expérimental

Même modèle, mêmes schémas, deux stratégies comparées sur des sample_id appariés.

5

JSON Schemas

200

échantillons / schéma

2

stratégies (le baseline et le retry)

2000

résultats finaux

Configuration du modèle

Provider	`OpenRouter`
Model	`qwen/qwen3.5-9b`
Temperature	`0.2`
Max output tokens	`4096`

Comparaison

BASELINE 1 tentative maximum

RETRY jusqu'à 3 tentatives + feedback

Important : baseline et retry sont des appels distincts au LLM. La comparaison porte sur les stratégies, pas sur la réparation des mêmes sorties.

Résultat principal

La stratégie retry élimine les échecs finaux observés dans la campagne baseline.

BASELINE

98.9%

conformité finale

989 / 1000 résultats valides

Final  **98.9%**

11 échecs finaux

RETRY

100%

conformité finale

1000 / 1000 résultats valides

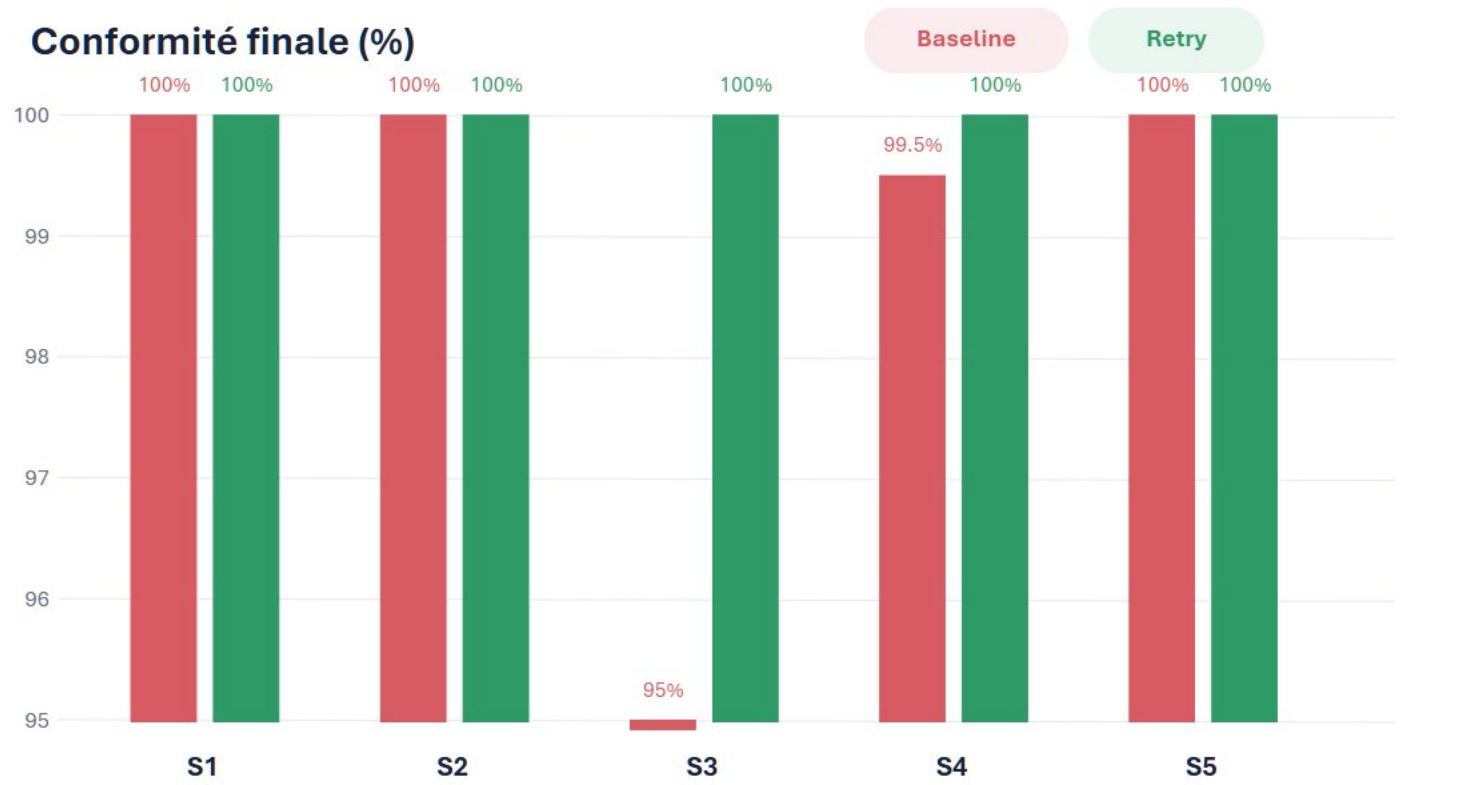
Final  **100%**

0 échec final

+1.1 point de conformité finale

Résultats par schéma

Le retry est surtout utile sur les schémas les plus difficiles.



FOCUS S3

95% → 100%

Baseline : 190 / 200
Retry : 200 / 200

1.04 tentative en moyenne

Dans le groupe retry, la conformité au premier essai est de 96% et atteint 100% après reprise.

Quel coût pour ce gain de fiabilité ?

Le retry améliore la conformité avec un surcoût et une hausse de latence très limités dans cette campagne.

Coût total

Baseline **$0.055469**

Retry **$0.056277**

+ $0.000808 ≈ +1.46%

→ faible au regard du passage à 100% de conformité finale

Latence moyenne



+44 ms ≈ +1.9%

La reprise n'est déclenchée que lorsque nécessaire.

Tests, reproductibilité et démonstration

53 passed

suite de tests automatisés

Mock

mode hors ligne

JSONL

résultats reproductibles

CLI

démo

Reproduire l'analyse

```
-$env:PYTHONPATH="src"

-python -m pytest -q

-python -m json_stress.cli evaluate results\experiment_final.jsonl

-python -m json_stress.cli report results\experiment_final.jsonl --
 output-dir results\report
```

Démo soutenance

- 1. Générer un JSON avec le MockGenerator
- 2. Montrer un échec de validation
- 3. Montrer le retry et le feedback
- 4. Afficher les métriques / le rapport final

Plan B : MockGenerator / résultats JSONL hors ligne

Limites et conclusion

Les résultats sont encourageants, mais ils restent liés au modèle et au protocole testés.

Limites

- Un LLM reste probabiliste
- Résultats dépendants du modèle et de la température
- Une autre API peut produire un comportement différent
- Certaines règles métier dépassent JSON Schema
- Retry = coût et latence supplémentaires

À généraliser sur d'autres modèles

CONCLUSION

Validation + feedback + retry borné

permet d'augmenter la robustesse d'un LLM lorsque l'application exige un contrat JSON strict.

98.9% → 100%

conformité finale

2000 résultats • 53 tests • démo hors ligne

Merci

Questions ?

A3 — Contrats JSON sous stress

Baseline 98.9%

Retry 100%

2000 résultats